ADR 0001 — Agent loop pattern

Status: Confirmed **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0002, docs/ARCHITECTURE.md, docs/DEFERRED.md

---

Context

BAYMAX's agent core needs a loop pattern that:

1. Takes a user request and produces a sequence of tool calls (single or multi-step)
2. Validates each step against the tool-call contract before execution
3. Recovers from errors (tool failures, ambiguous intent, model hallucinations) without silently failing
4. Supports clarification flows when the model can't determine the user's intent

The choice of loop pattern affects every downstream module: telemetry shape, scoring boundary with Ronin's eval harness, recovery semantics, and CV defensibility under "walk me through how your agent works."

The literature offers three canonical patterns:

- **ReAct** (Reason-Act-Observe loop): the model reasons, picks one action, observes the result, and loops until done
- **Plan-and-Execute**: the model produces a full plan upfront, then an executor runs each step in sequence
- **Tool-use chat loop** (OpenAI / Anthropic native): the model emits tool calls; the orchestrator dispatches and feeds results back as new messages until the model emits a final response

---

Decision

BAYMAX uses **Plan-and-Execute with a finite iterative ReAct-style loop for error and clarification recovery**.

Concretely:

1. The model interprets the user request and constructs a `task_file` (a plan containing the sequence of intended tool calls, response type, etc.)
2. The `task_file` is validated against the tool-call contract (ADR 0002) before any tool fires
3. The executor dispatches each tool call in the planned sequence; results stream back as the action log grows
4. If the executor hits an error (tool failure, invalid argument, ambiguous result, timeout), control passes to a recovery loop that can:
    - Retry the failing step (with backoff)
    - Ask the user for clarification (terminating the request for v1; sub-tasks deferred to v2)
    - Trigger an abort with rollback of completed-and-reversible actions

5. The model is also invoked at the end to interpret raw tool outputs into a user-facing response message

Pseudocode (≤30 lines):

```
function handle_request(user_input):
    task_file = model.construct_task_file(user_input, available_tools, context)
    validation = validator.validate(task_file)
    if not validation.ok:
        return error_response(validation.errors)

    telemetry.start_trace(task_file.task_id)
    action_log = []

    for step in task_file.tool_calls:
        try:
            result = executor.dispatch(step)
            action_log.append((step, result))
            if result.requires_clarification:
                return clarification_response(result.question, action_log)
        except RecoverableError as e:
            recovery = recovery_handler.handle(e, action_log, task_file)
            if recovery.action == "retry":
                continue
            elif recovery.action == "clarify":
                return clarification_response(recovery.question, action_log)
            elif recovery.action == "abort":
                rollback(action_log)
                return abort_response(e, action_log)
        except IrreversibleError as e:
            return failure_response(e, action_log)

    response_text = model.interpret_results(action_log, user_input)
    return success_response(response_text, action_log)
```

Reasoning

why chosen - allows for predictability of upfront planning (auditable and reviewable before any side effect) PLUS resilience of ReAct-style

recovery for issue catching and gracefully terminating where needed. (real tool calls fail, but the agent shouldn't crash essentials).

Alternatives considered

Alternative A — Pure ReAct

The model would emit one tool call at a time, observe the result, and decide the next step in a single loop without an upfront plan.

why rejected - harder to audit before side effects fire; model context fills up with redundant reasoning across many steps; no separation between planning errors and execution errors which makes telemetry less useful.

Alternative B — Pure Plan-and-Execute (no recovery loop)

The model would emit a full plan upfront; the executor runs every step in sequence without a path back to the model on failure.

why rejected - tools sometimes fail (5xx, rate limits, auth expiry); without a recovery path the agent crashes or returns broken state to the user; production agents must handle failure as a first-class case, especially considering the rigorous tests we plan to use the agent for in evaluation.

Alternative C — Tool-use chat loop (OpenAI / Anthropic native)

The model emits tool calls via the SDK's native function-calling interface; the orchestrator dispatches and feeds results as new messages until the model emits a final response.

why rejected - tightly linked to one provider's tool-use protocol; harder to swap inference backends (Qwen, local MLX) without rewriting the loop; loses the validation-before-execution discipline.

Consequences

Positive

- Plans are auditable BEFORE any side effect fires (the `task_file` exists and is logged before tool dispatch)
- Telemetry has a natural artifact (the task_file) to attach traces to
- Recovery is explicit and traceable, not silent retries hidden inside the model
- Boundary with Ronin's eval is clean: agent emits a structured `AgentResponse` matching `scorer.py` shape

Negative

- Two model invocations per request (one to construct the plan, one to interpret results) — higher cost per request than pure ReAct on simple cases
- Plan-and-Execute is less flexible when the right next action depends on the result of the previous one — the model has to either over-plan (anticipate branches) or trigger replanning via the recovery loop
- More moving parts than a pure tool-use chat loop — more surface area to debug

Neutral

- Multi-turn user sessions (continuation across requests) are not addressed by this loop; deferred to v2 sub-task model (see DEFERRED.md)

Open questions / follow-ups

- **Sub-task chains for clarification follow-ups** (1.0 → 1.1 model) — deferred to v2 per DEFERRED.md. Current pattern terminates on clarification; user manually resubmits.
- **Streaming response** during the "interpret results" phase — deferred to v2 (see DEFERRED.md). v1 returns all-at-once.
- **Multi-phase tasks** for irreversible tools (draft-then-send) — deferred to v2 with multi-phase implementation choice (versioned task_File vs sub-task chain) still open.

References

- ReAct paper: Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models" (2022)
- Plan-and-Execute style: LangChain "Plan-and-Execute Agents" docs
- OpenAI tool-use: https://platform.openai.com/docs/guides/function-calling
- Ronin's `AgentResponse` shape (boundary contract): `src/baymax/eval/scorer.py` on branch `feat/phase1-scenario-loader`
- docs/WRITEUP_INFO.md for the broader thesis BAYMAX is testing

---

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.